Clase repaso

Programación II

5

Noviembre

UNRN

Universidad Nacional
de **Río Negro**

r27

Programación I

Repaso

Universidad Nacional de Río Negro | Ingeniería en Sistemas | Ciclo 2026

Prof. Daniel Teira/Diego Diaz

Sede Bariloche

Repaso: stack

1. Un Stack (Pila) funciona como LIFO (Last-In, First-Out / Último en entrar, Primero en salir). El ejemplo clásico es una pila de platos: el último plato que pones arriba es el primero que sacas.
2. Una Queue (Cola) funciona como FIFO (First-In, First-Out / Primero en entrar, Primero en salir). El ejemplo es la fila de un supermercado: el primero en llegar es el primero en ser atendido.
3. En el siguiente ejemplo tenemos:
    - `inicializar(Pila *p)`: Configura el estado inicial de la pila colocando el índice `tope` en `-1` (vacía).
    - `empty(Pila *p)`: Retorna `true` si la pila está completamente vacía (`tope == -1`).
    - `full(Pila *p)`: Retorna `true` si la pila alcanzó su capacidad máxima y no admite más elementos.
    - `push(Pila *p, int numeroPlato)`: Apila un nuevo elemento en el tope, siempre y cuando no esté llena.
    - `pop(Pila *p)`: Extrae, elimina lógicamente (restando el tope) y retorna el elemento superior, verificando antes que no esté vacía.
    - `top(Pila *p)`: Retorna el elemento superior para inspeccionar sin modificar el estado de la pila (solo lectura).

Repaso: stack

```c
#include <stdio.h>
#include <stdbool.h>

#define MAX_PLATOS 5 // Límite pequeño para forzar la prueba de Pila Llena

// Definición de la estructura Pila
typedef struct {
    int platos[MAX_PLATOS];
    int tope;
} Pila;
```

```c
// 1. Inicializar la pila
void inicializar(Pila *p) {
    p->tope = -1;
}

// 2. Consultar si está vacía
bool empty(Pila *p) {
    return p->tope == -1;
}

// 3. Consultar si está llena
bool full(Pila *p) {
    return p->tope == MAX_PLATOS - 1;
}
```

Repaso: stack

```
// 4. Operación PUSH (Apilar)
void push(Pila *p, int numeroPlato) {
    if (full(p)) {
        printf("[ERROR PUSH] La pila está llena. No cabe el
plato %d.\n", numeroPlato);
    } else {
        p->tope++;
        p->platos[p->tope] = numeroPlato;
        printf("[PUSH] Plato %d apilado en la posicion
%d.\n", numeroPlato, p->tope);
    }
}
```

```
// 5. Operación POP (Desapilar)
int pop(Pila *p) {
    if (empty(p)) {
        printf("[ERROR POP] La pila está vacía. Nada que
extraer.\n");
        return -1; // Valor centinela o de error
    } else {
        int platoSacado = p->platos[p->tope];
        p->tope--; // Eliminación lógica
        printf("[POP] Plato %d extraido.\n", platoSacado);
        return platoSacado;
    }
}
```

Repaso: stack

```
// 6. Consultar el tope (Lectura sin
extraer)
int top(Pila *p) {
    if (empty(p)) {
        printf("[ERROR TOP] La pila
está vacía. No hay elementos para
leer.\n");
        return -1;
    }
    return p->platos[p->tope];
}
```

Repaso: stack

```
int main() {
    Pila miPila;
    inicializar(&miPila);

    printf("=== DEMOSTRACION DE STACK (LIFO) ===\n\n");

    printf("1. APILANDO ELEMENTOS (PUSH)\n");
    push(&miPila, 101);
    push(&miPila, 102);
    push(&miPila, 103);

    printf("\n2. CONSULTANDO EL TOPE (TOP)\n");
    printf("-> El elemento en el tope es: %d\n", top(&miPila));
    printf("-> Consulto de nuevo, sigue siendo: %d\n", top(&miPila));

    printf("\n3. EXTRACCION Y MUTACION (POP)\n");
    pop(&miPila);
    printf("-> Despues del POP, el nuevo tope es: %d\n", top(&miPila));
```

```
    printf("\n4. LLENANDO LA PILA AL MAXIMO\n");
    push(&miPila, 104);
    push(&miPila, 105);
    push(&miPila, 106); // Esto debería apilarse bien (van 5)
    push(&miPila, 107); // Esto debería dar error de Pila Llena

    printf("\n5. VACIANDO LA PILA\n");
    while (!empty(&miPila)) {
        pop(&miPila);
    }
    pop(&miPila); // Esto debería dar error de Pila Vacía

    return 0;
}
```

Repaso: stack comentarios

1. Por qué “-1”? porque los arrays en C empiezan en 0). Cuando entra el primer elemento, el tope suma 1 y queda en 0.
2. No se borran los datos en memoria al hacer POP: Solo se resta el índice tope. Si imprimieran el arreglo completo ignorando el tope, el "103" seguiría ahí en la RAM, simplemente ahora se considera "basura" que será sobrescrita en el próximo Push.
3. Seguridad: Destacar la importancia de las funciones ***full*** y ***empty*** para evitar que el programa colapse (Segmentation Fault).

Repaso: Ejercicio

1. Gestionar la pila, pero no usar int, si no la pila de un texto (cadena de texto)